

LOCALIZAÇÃO

Desenvolvimento do App TechCall: Checkin com coordenadas e Reverse Geocoding

11 de agosto de 2026

1. VISÃO GERAL DA AULA

Na Aula anterior, o técnico do **TechCall** passou a conseguir registrar a evidência fotográfica de um chamado. Hoje, o app ganha "consciência geográfica": ao abrir um novo chamado, o técnico poderá capturar sua localização atual e convertê-la automaticamente em um endereço legível, simulando o check-in no local do atendimento.

Ao final desta aula, o aluno será capaz de:

1. Solicitar e tratar permissões de localização em tempo de execução.
2. Obter a localização atual do dispositivo (latitude e longitude) com **expo-location**.
3. Converter coordenadas em um endereço legível (*Reverse Geocoding*).
4. Exibir estados de carregamento (loading) durante operações assíncronas de GPS.
5. Tratar cenários de erro: GPS desligado, permissão negada, sinal demorado.

Materiais Necessários

- Computador (Windows ou macOS) com VS Code instalado.
- Celular com o aplicativo **Expo Go** instalado (disponível na Play Store ou App Store) e GPS ativo.
- Projeto techcall criado na Aula anterior, com o Expo Go configurado e funcionando.

2. PREPARAÇÃO

Diferente da câmera, o recurso de localização depende de sinal de GPS, que costuma ser mais lento (ou instável) dentro do prédio. Pode ser necessário se movimentar para buscar locais com boa recepção de sinal perto de janelas. A primeira captura de localização pode demorar alguns segundos.

3. INSTALANDO A BIBLIOTECA DE LOCALIZAÇÃO

Assim como fizemos com a câmera, vamos instalar um módulo Expo específico para localização. No terminal, dentro da pasta techcall, digite:

```
npx expo install expo-location
```

4. CONFIGURANDO PERMISSÕES

Diferente da câmera (que o expo-image-picker já configura automaticamente), a localização exige que declaremos o motivo do uso do GPS no arquivo app.json — isso é obrigatório para publicação nas lojas e já vale a pena configurar agora.

Abra o arquivo **app.json** na raiz do projeto e adicione o bloco plugins dentro de expo:

```
{
  "expo": {
    "name": "techcall",
    "slug": "techcall",
    "plugins": [
      [
        "expo-location",
        {
          "locationAlwaysAndWhenInUsePermission": "Permita o acesso à localização para registrar o local do chamado."
        }
      ]
    ]
  }
}
```

ATENÇÃO! O app.json já deve ter outras chaves (name, version, icon etc.), apenas acrescente a chave plugins no mesmo nível — **não substitua o arquivo inteiro**.

5. CÓDIGO GUIADO (APP TECHCALL)

Vamos ampliar o NewCallScreen.tsx da aula anterior. O técnico poderá tocar em "Registrar localização" para capturar as coordenadas do chamado e ver o endereço convertido na tela, com um indicador de carregamento durante a busca.

Etapa 1: Importações e novos states

No topo do arquivo src/screens/NewCallScreen.tsx, adicione a importação do módulo de localização:

```
import * as Location from 'expo-location';
```

Dentro do componente, logo após os states já existentes (description e photoUri), adicione:

```
// Endereço já convertido, pronto para exibir na tela
const [address, setAddress] = useState<string | null>(null);
// Controla o texto "Buscando localização..." enquanto aguardamos o GPS
const [loadingLocation, setLoadingLocation] = useState(false);
```

Etapa 2: Função de captura de localização

Ainda no NewCallScreen.tsx, adicione a função abaixo (pode ficar logo após handlePickFromGallery):

```
async function handleGetLocation() {
  // 1. Pede permissão de localização em primeiro plano
  const permission = await Location.requestForegroundPermissionsAsync();

  if (!permission.granted) {
    Alert.alert('Permissão negada', 'Precisamos da localização para o check-in.');
```

return;

```
  }

  setLoadingLocation(true);

  try {
    // 2. Verifica se o GPS do aparelho está ligado
    const gpsAtivo = await Location.hasServicesEnabledAsync();
    if (!gpsAtivo) {
      Alert.alert('GPS desligado', 'Ative a localização do aparelho e tente novamente.');
```

return;

```
    }

    // 3. Captura latitude e longitude atuais
    const position = await Location.getCurrentPositionAsync({
      accuracy: Location.Accuracy.Balanced,
    });

    // 4. Reverse Geocoding: transforma coordenadas em endereço legível
    const [local] = await Location.reverseGeocodeAsync({
      latitude: position.coords.latitude,
      longitude: position.coords.longitude,
    });

    if (local) {
      const enderecoFormatado = `${local.street ?? 'Endereço não identificado'}, ${local.city ?? ''} - ${local.region ?? ''}`;
      setAddress(enderecoFormatado);
    } else {
      setAddress('Endereço não encontrado para esta coordenada.');
```

}

```
  } catch (error) {
    Alert.alert('Erro', 'Não foi possível obter a localização. Tente novamente.');
```

}

```
  } finally {
    setLoadingLocation(false);
  }
}
```

Etapa 3: Exibindo o resultado na tela

No JSX do NewCallScreen (dentro do ScrollView, logo após o bloco de foto e antes do buttonRow), adicione a seção de localização:

```
<Text style={styles.label}>Localização do chamado</Text>

{loadingLocation ? (
  <Text style={styles.placeholderText}>Buscando localização...</Text>
) : address ? (
  <Text style={styles.addressText}>{address}</Text>
) : (
  <Text style={styles.placeholderText}>Nenhuma localização registrada</Text>
)}

<TouchableOpacity
  style={[styles.button, styles.locationButton]}
  onPress={handleGetLocation}
  disabled={loadingLocation}
>
  <Text style={styles.buttonText}>
    {loadingLocation ? 'Buscando...' : 'Registrar localização'}
  </Text>
</TouchableOpacity>
```

E, no StyleSheet.create ao final do arquivo, acrescente os dois novos estilos usados acima:

```
addressText: { fontSize: 14, color: '#333', marginBottom: 8 },
locationButton: { backgroundColor: '#00695c', marginBottom: 16 },
```

6. EXPLICAÇÃO DOS CONCEITOS

1. **Coordenadas (lat/long):** são o "CEP numérico" do planeta — todo ponto na Terra tem uma latitude e uma longitude únicas. É o dado bruto que o GPS devolve.
 2. **Reverse Geocoding:** é o processo de traduzir coordenadas em algo legível para humanos (rua, bairro, cidade) — o "carteiro" que transforma números em endereço.
 3. **Accuracy (precisão):** quanto maior a precisão pedida, mais tempo e bateria o GPS consome. Balanced é um bom equilíbrio para a maioria dos apps.
 4. **hasServicesEnabledAsync:** verifica se o GPS do aparelho está fisicamente ligado — diferente da permissão, que é sobre o app ter autorização para usá-lo.
 5. **try/catch/finally:** usamos porque a busca de localização pode falhar (sinal fraco, timeout). O finally garante que o indicador de carregamento sempre será desligado, tenha dado certo ou não.
-

7. ERROS COMUNS E SOLUÇÕES

Sintoma	Causa Provável	Solução
Botão fica em "Buscando..." para sempre	Sinal de GPS fraco (dentro de prédio) ou setLoadingLocation não é chamado no finally	Testar perto de uma janela; conferir se o finally está presente na função
Endereço aparece como "undefined"	Uso de local.street sem tratamento de campo ausente	Usar o operador ?? para fornecer um valor padrão, como no exemplo
App trava ao tocar no botão de localização	Falta de app.json configurado, causando erro nativo silencioso	Conferir o bloco plugins no app.json e reiniciar o servidor Expo
Permissão não é solicitada	App já foi negado antes e o sistema não pergunta de novo	Ir em Configurações do celular > Expo Go > Localização e permitir manualmente